Closing the Tag, 0wning the Agent: Tag Boundary Injection in Bedrock Agent Orchestration

Paper Abstract

LLM agents are no longer just chat interfaces. They plan, call tools, query knowledge bases, and drive workflows. Amazon Bedrock packages all of this into “Agents” - a clean abstraction over models, orchestration, and integrations. That abstraction is useful, but it hides an untapped attack surface: the **agent orchestration layer**.

This talk looks at Bedrock Agents from the inside out. Using traces and callbacks, we break down how an agent actually runs:

- Pre-processing templates that normalize input
- Orchestration templates that implement a ReAct-style loop
- Knowledge base response generation
- Optional post-processing
- Memory summarization
- Routing/classifier layers

The key observation: Bedrock treats model output as structured control data, not just text. Tags like <thinking> and <answer> are part of how the runtime decides what to do next - whether to call a tool, continue reasoning, or return a response.

These tags are not enforced by a strict parser. They’re generated by the model and interpreted by the runtime. That means user-controlled input can influence how these boundaries are formed and how they are read.

This is where **tag boundary injection** comes in.

Instead of trying to “jailbreak” the model, the focus shifts to **breaking the contract between model output and the orchestration layer**.

We explore a set of attack primitives:

- Prematurely closing boundary tag blocks to alter how responses are handled
- Injecting into thinking loops to influence reasoning and tool selection
- Breaking tag boundaries inside tool parameters to cause parsing ambiguity
- Injecting via knowledge base or tool outputs to influence future steps

The interesting cases are indirect. If external data (KB results, API responses) is fed back into the orchestration loop, it becomes part of the next prompt. That gives you a way to inject into the system without touching the original user input.

The demos walk through these primitives and show potential malicious outcomes:

- Forcing the agent to skip intended tool flows
- Manipulating how decisions are made across steps
- Confusing the runtime about what is “final output” vs intermediate state
- Divulging internal order and confidential prompt data and variables

The attack surface has moved up the stack. It’s no longer just “what the model says,” but **how its output is parsed, trusted, and acted on**.

Presentation Time Required

30 minutes

Category

Category 1

Is there any demonstration?

Yes

Are you releasing any new tools?

No.

Are you releasing any new exploits?

Yes

Have you submitted / presented the paper before at any other security / technology conference(s)?

No.

Presentation Track

AI / Other Research